

Module 2: Web Architecture and Technology

Jayvee Christopher N. Vibar, DIT

Module Overview

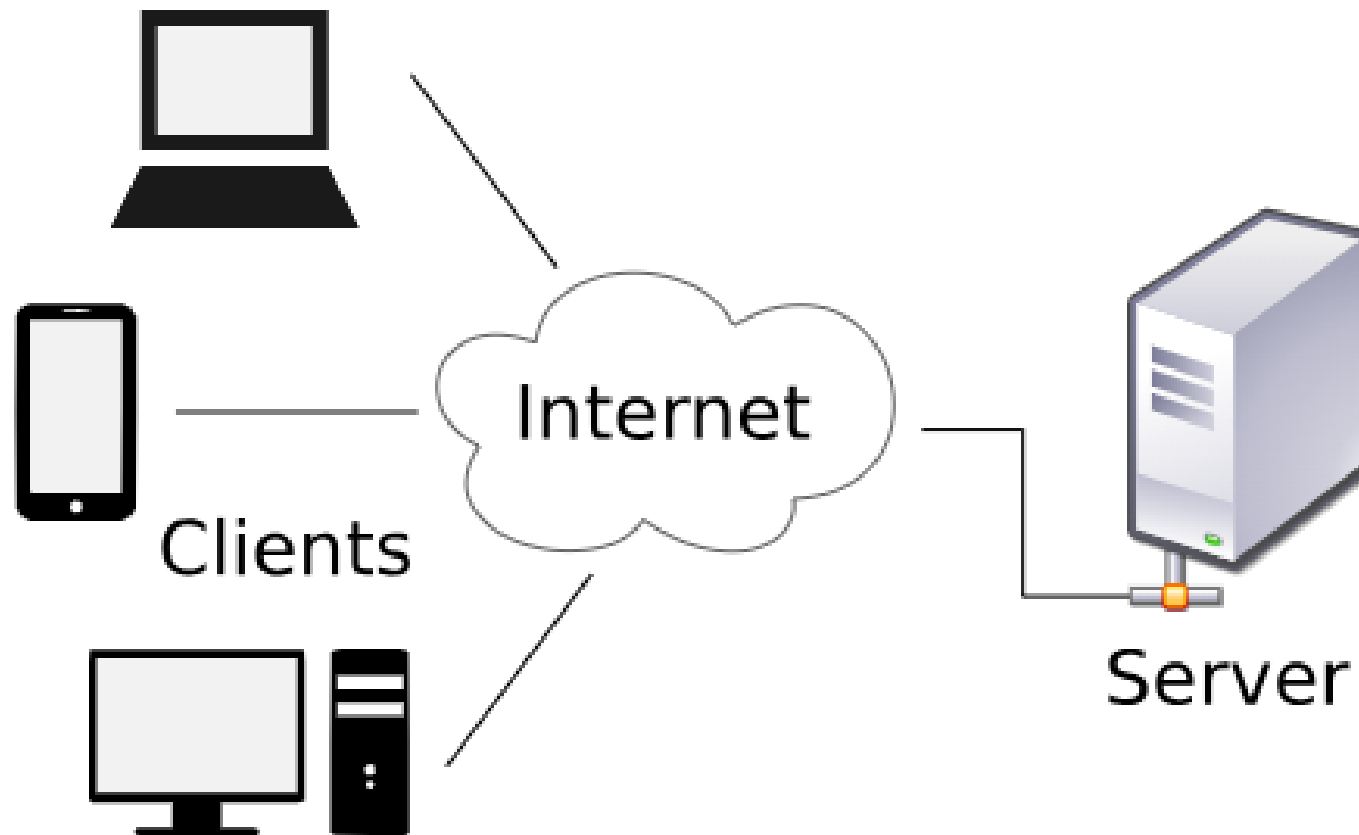
- What is Web Architecture?
- Why architecture matters in modern web systems
- How this module connects to real-world systems (LMS, e-commerce, portals)

- **Web architecture** is the conceptual framework that determines how the internet's components—servers, databases, and browsers—interact with one another. Think of it as the blueprint for how data travels from a remote server to your screen.

The Core Components

Modern web architecture is generally built on a **Client-Server Model**.

- **The Client:** This is the "front end." It's the device or application a user interacts with (like Chrome, Safari, or a mobile app). It sends requests to the server.
- **The Server:** This is the "back end." It's a powerful computer that sits in a data-center, waiting for requests. It processes the logic and sends back the necessary files.
- **The Database:** The storage room. This is where the server retrieves or saves persistent data.



2. How the Request Flow Works

To understand the architecture, it helps to follow the journey of a single click:

1. **DNS Request:** Your browser looks up the "IP address" of the website name (converting `google.com` into a string of numbers like `142.250.190.46`).
2. **HTTP Request:** The browser sends a formal request to that IP address via the **HTTP/HTTPS** protocol.
3. **Processing:** The Web Server receives the request. If the site is "dynamic" (like Facebook), it talks to an **Application Server** to run code and a **Database** to pull your specific feed.
4. **Response:** The server sends back a "Response" containing HTML, CSS, and JavaScript.
5. **Rendering:** Your browser parses those files and turns them into the visual website you see.

3. Layers of Modern Architecture

As websites grow into massive "Web Apps," the architecture gets more sophisticated to handle millions of users:

Layer	Function	Tools
User Interface	What the user sees and clicks.	HTML, CSS, React, <u>Vue</u>
Business Logic	The "brain" that decides what happens when you click.	Node.js, Python, Java
Data Layer	Where information is organized and stored.	MySQL, <u>PostgreSQL</u> , <u>MongoDB</u>
Infrastructure	The physical or virtual hardware.	AWS, Azure, Google Cloud

4. Why Does It Matter?

Good architecture isn't just about making things work; it's about making them **scalable** and **secure**.

- **Scalability:** Can the site handle 10 users today and 10 million tomorrow? (Usually handled by **Load Balancers**).
- **Reliability:** If one server crashes, does the whole site go down?
- **Security:** How do we ensure a "Middle Man" doesn't intercept your credit card data? (Handled by **SSL/TLS encryption**).

Evolution of Web Systems

- Static websites (HTML only)
- Dynamic websites (PHP, ASP, JSP)
- Web applications
- Cloud-based and distributed systems

Basic Web Communication

- Browser (Client)
- Web Server
- Database Server
- HTTP/HTTPS request–response cycle

Key Architecture Goals

- Scalability
- Performance
- Security
- Maintainability
- Availability

Q & A

“What web systems do you use daily and what problems might they face if badly designed?”

Client–Server Architecture

Client-Server Architecture is a distributed computing structure that partitions tasks or workloads between the providers of a resource or service (the **servers**) and the service requesters (the **clients**).

Unlike a peer-to-peer network where everyone is equal, this is a hierarchical relationship designed to centralize data and distribute access.

2-Tier Architecture

- Client $\leftrightarrow$ Server
- Examples:
 - Desktop app + database
 - Simple PHP + MySQL setup

Strengths and Limitations

Pros

- Simple
- Easy to develop

Cons

- Poor scalability
- Security risks
- Performance bottlenecks

Real-World Example

- Login system
- Simple school portal

Multi-Tier (N-Tier) Architecture

What is Multi-Tier Architecture?

- Separation of concerns
- Layers with specific responsibilities

3-Tier Architecture

- Presentation Layer (UI)
- Application Layer (Business Logic)
- Data Layer (Database)

N-Tier Architecture

Components:

- API layer
- Authentication services
- Cache layer
- Load balancers

Benefits of Multi-Tier

- Scalability
- Easier maintenance
- Better security
- Team-based development

Example Systems

- Online banking
- University information systems
- E-commerce platforms

Q & A

- Identify tiers in Facebook / Shopee / GCash / Google Classroom

Thank You